UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería Industrial y de Sistemas
SI806 · Desarrollo Adaptativo e Integrado del Software

TALLER 3 · DESARROLLO E INTEGRACIÓN DE UNA API REST

Informe de Demostración

Cliente móvil consumiendo la API REST y cambios persistidos en la base de datos

Evidencia ordenada del recorrido completo de Luxxito Sports ejecutado el 3 de noviembre de 2026 con el APK de producción en un celular Android físico, la API pública en Render y la base de datos PostgreSQL de Supabase.

Integrantes

Steven Cadillo
Cliente móvil, pagos, pruebas y demostración

Elvis Arboleda
Base de datos Supabase y despliegue

Juan Medina
Backend, Render e infraestructura

Repositorio en GitHub (rama main)
`https://github.com/Arbolito-ctrl/LuxxitoSports/tree/main`

API pública
`https://luxxito-sports-api.onrender.com/api`

Lima, noviembre de 2026

Índice

1. Introducción 2
2. Entorno y datos de la prueba 2
3. Desarrollo secuencial de la demostración 3
3.1. Catálogo y filtros 3
3.2. Disponibilidad y cotización calculada por el servidor 3
3.3. Creación de la reserva 3
3.4. Exclusión mutua: el mismo turno no se vende dos veces 3
3.5. Pago con foto del comprobante 4
3.6. Revisión y aprobación del administrador 4
3.7. Ingreso al complejo con el Ticket QR 5
3.8. Cancelación con la política de 24 horas 6
4. Verificación de la persistencia en Supabase 7
5. Resumen de resultados 8
6. Conclusiones 8

1. Introducción

El Taller 3 exige demostrar a un **cliente consumiendo la API propia**, con una **operación completa** y su **cambio persistido en la base de datos**. Este informe registra esa demostración paso a paso: para cada etapa se indica qué se hizo en la aplicación, qué endpoint respondió y qué quedó guardado en Supabase, acompañado de las capturas tomadas en el celular.

El recorrido cubre las seis reglas de negocio del servidor: catálogo y disponibilidad, cotización con tarifa diurna y nocturna, reserva con bloqueo de 20 minutos y exclusión mutua, pago con comprobante en revisión, aprobación del administrador con emisión del Ticket QR de un solo uso y cancelación con la política de 24 horas.

2. Entorno y datos de la prueba

Tabla 1. *Entorno de ejecución de la demostración.*

Componente	Detalle
Fecha y hora	3 de noviembre de 2026, de 15:38 a 15:54 (hora de Lima)
Cliente	APK de producción 1.0.36 (Flutter) instalado en un Motorola Edge 50 con Android 16
API	Backend propio Node.js + Express + TypeScript en Render: https://luxxito-sports-api.onrender.com/api
Base de datos	Supabase PostgreSQL mediante el pooler de transacciones (db_mode = POSTGRES_POOL) y Storage con el bucket privado "comprobantes"
Usuarios	Una cuenta de rol JUGADOR y una cuenta de rol ADMIN (operador del complejo)

Tabla 2. *Datos utilizados en la demostración.*

Dato	Valor
Cancha	Cancha Monumental 1 · Futbol 7 · Luxxito Arena San Miguel
Turno	2026-11-05, de 17:00 a 19:00 (1 hora diurna + 1 hora nocturna)
Extras	2 juegos de chalecos (S/ 15 cada uno)
Total esperado	S/ 90 (día) + S/ 120 (noche) + S/ 30 (chalecos) = S/ 240.00; seña del 50% = S/ 120.00
Registros creados	Reserva #19, pago #5, reserva del competidor #20 y cancelación de la reserva #4

3. Desarrollo secuencial de la demostración

3.1. Catálogo y filtros

El jugador inicia sesión y en **Explorar** filtra por distrito (San Miguel) y deporte (Futbol 7). La aplicación no guarda datos locales: la lista proviene de `GET /api/complejos` y `GET /api/canchas?distrito=…&deporte=…`, que consultan PostgreSQL. **Resultado:** `200 OK`, solo aparecen canchas de Futbol 7 en San Miguel, con foto y precio.

3.2. Disponibilidad y cotización calculada por el servidor

Se elige la Cancha Monumental 1, la fecha y los turnos de 17:00 y 18:00, y se agregan 2 juegos de chalecos. Los horarios vienen de `GET /api/canchas/1/disponibilidad` y el precio lo calcula el servidor con `POST /api/reservas/cotizar`: desde las 18:00 rige la tarifa nocturna. **Resultado:** cancha S/ 210.00 + extras S/ 30.00 = **S/ 240.00**, adelanto del 50% S/ 120.00.

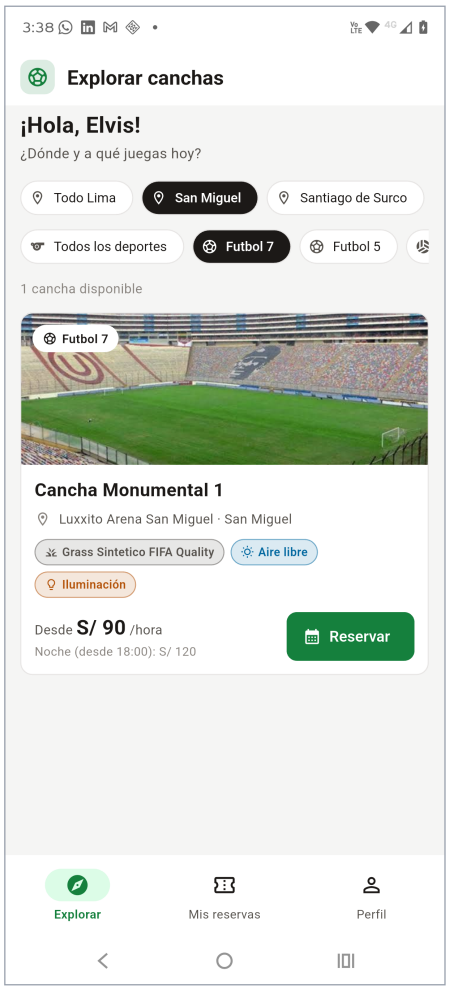


Figura 1
Catálogo filtrado por distrito y deporte.

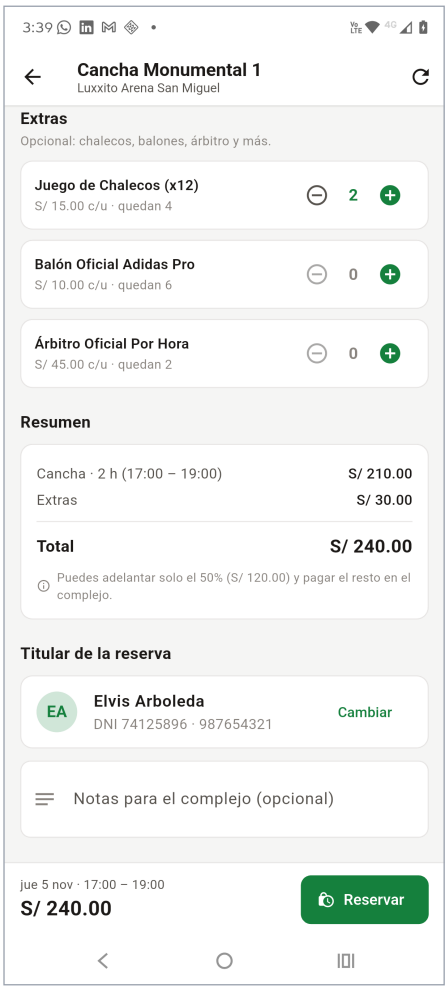


Figura 2
Cotización calculada por el servidor.

3.3. Creación de la reserva

`POST /api/reservas` responde **201 Created**. El servidor valida horario, solapamiento y stock de extras, y bloquea el turno 20 minutos. **Persistencia:** en `reservas`, la fila #19 con `estado = PENDIENTE_PAGO`, `total = 240` y `expires_at`; en `reserva_extras`, chalecos × 2 con subtotal S/ 30.

3.4. Exclusión mutua: el mismo turno no se vende dos veces

Al volver a la cancha, los turnos de 17:00 y 18:00 aparecen como *Reservándose*. Luego se simula a dos clientes compitiendo por las 20:00: un cliente externo reserva primero (reserva #20) y, cuando el jugador confirma el mismo turno, el servidor responde **409 Conflict** y la app explica que el horario ya no está libre.

En la base existe **una sola** reserva activa por horario.

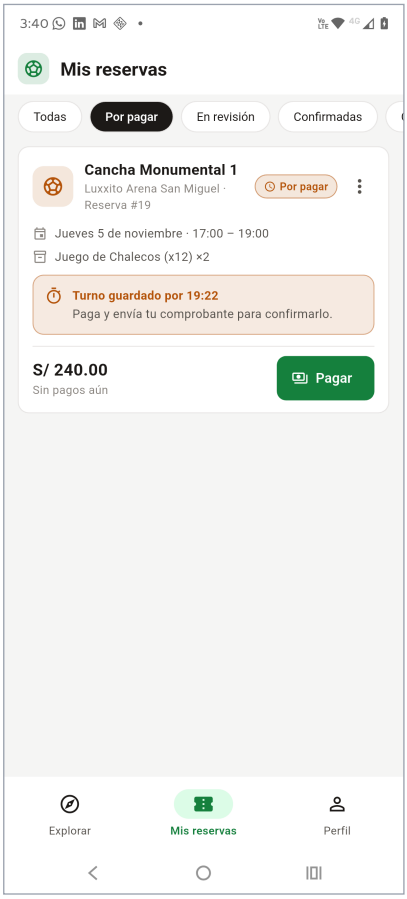


Figura 3
Reserva creada con bloqueo de 20 minutos.

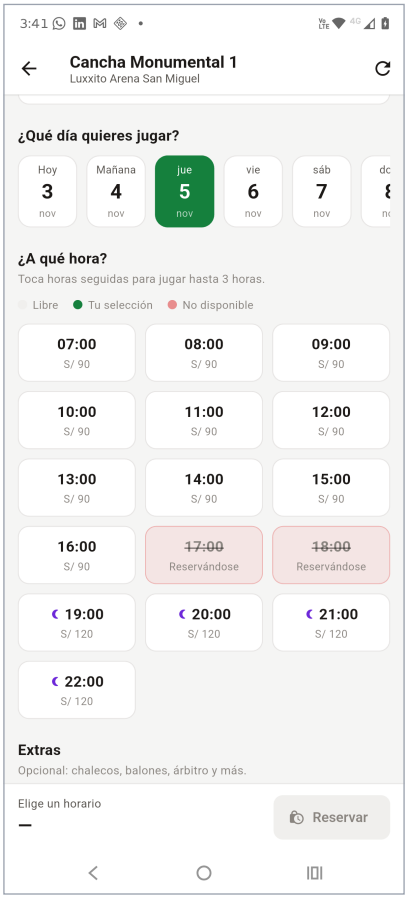


Figura 4
Turnos 17:00 y 18:00 bloqueados.

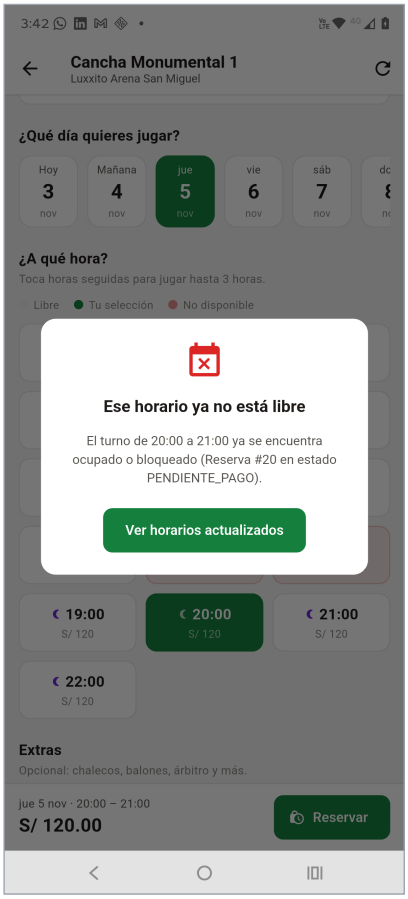


Figura 5
Respuesta 409 ante el turno ya tomado.

3.5. Pago con foto del comprobante

En **Mis reservas** el jugador elige *Pago total* con Yape, escribe el número de operación y adjunta la foto del voucher. La foto se sube al bucket privado con `POST /api/pagos/comprobante` y el pago se registra con `POST /api/reservas/19/pagos` (**201 Created**). **Persistencia:** `pagos` #5 en `PENDIENTE_REVISION` con `numero_operacion = YAPE-8492014`, el archivo en Storage y la reserva en `EN_REVISION`, que ya no vence.

3.6. Revisión y aprobación del administrador

El ADMIN tiene su propio panel. **Inicio** indica un pago por revisar y la pestaña **Pagos** muestra el voucher mediante un enlace firmado de 1 hora. Al aprobar, `PATCH /api/pagos/5/aprobar` (solo rol ADMIN; un JUGADOR recibe 403) confirma la reserva y emite el Ticket QR. **Persistencia:** pago `APROBADO` con `reviewed_at`; reserva `CONFIRMADA`, `monto_pagado = 240` y `codigo_qr = LUXXITO-QR-19-EAFECA58D7`.

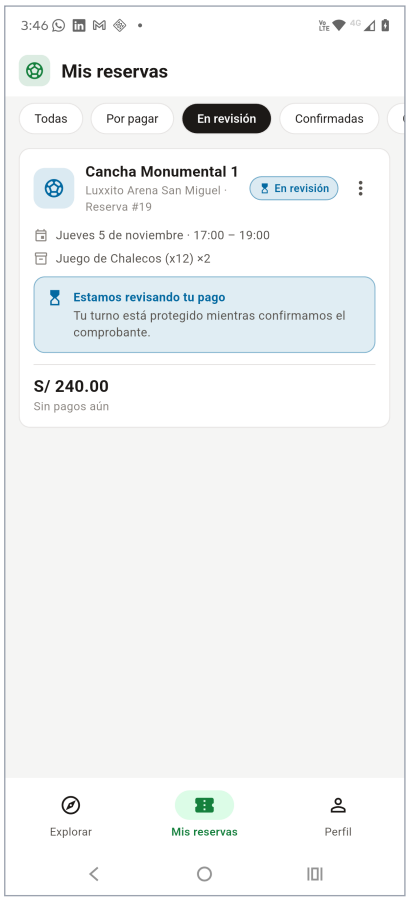


Figura 6
Pago enviado: reserva en revisión.

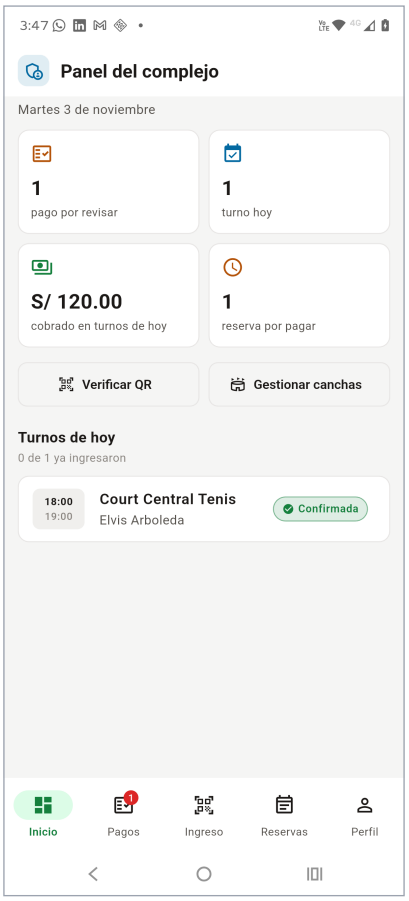


Figura 7
Panel del ADMIN con el pago pendiente.

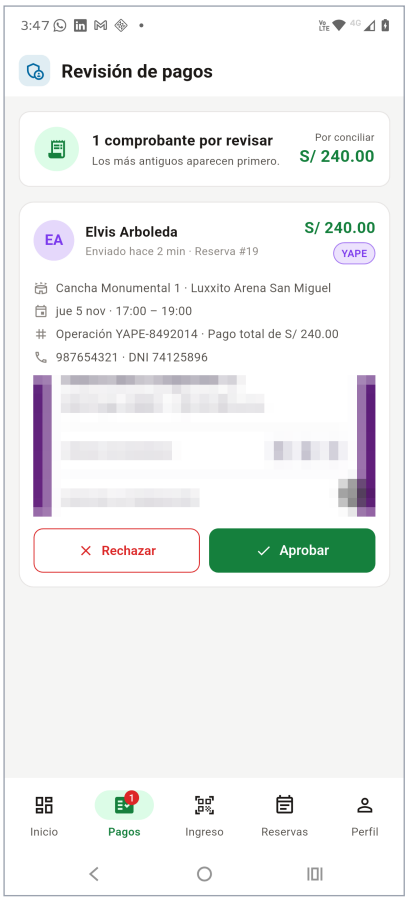


Figura 8
Bandeja de pagos con el voucher (difuminado).

> *En la Figura 8 el voucher está difuminado porque la imagen de prueba contenía datos de otra persona. El archivo original permanece en el bucket privado y solo el ADMIN puede verlo.*

3.7. Ingreso al complejo con el Ticket QR

El jugador muestra su Ticket y el ADMIN lo escanea en la pestaña **Ingreso**. `POST /api/reservas/verificar-qr` registra el ingreso una sola vez (**200 OK**, `qr_usado_at` = 15:50). Al escanear el mismo QR otra vez, la API responde **409**: "Este Ticket QR ya fue utilizado para ingresar". La marca se guarda con una operación atómica en PostgreSQL, por lo que ni dos lecturas simultáneas permiten entrar dos veces.

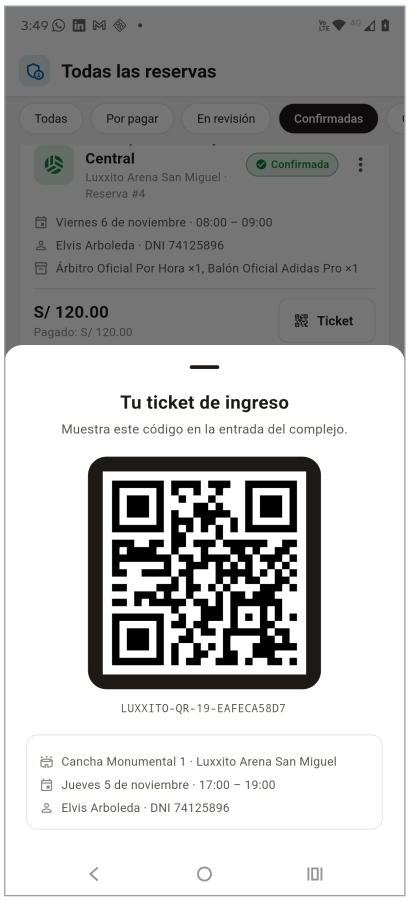


Figura 9
Ticket QR de la reserva confirmada.

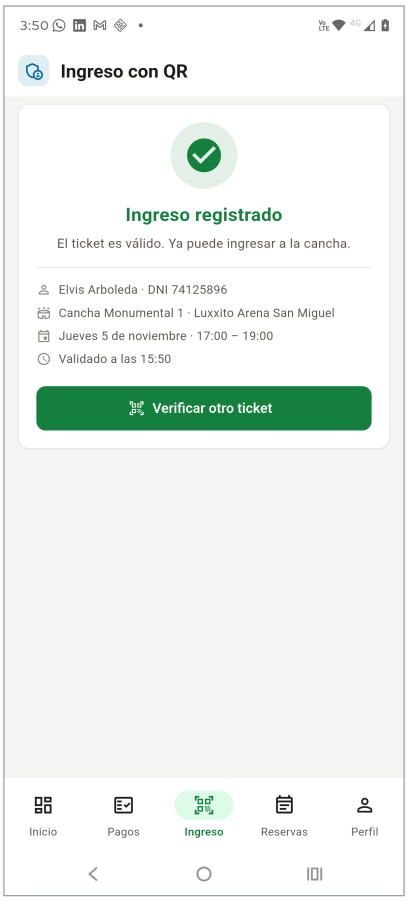


Figura 10
Primer escaneo: ingreso registrado.

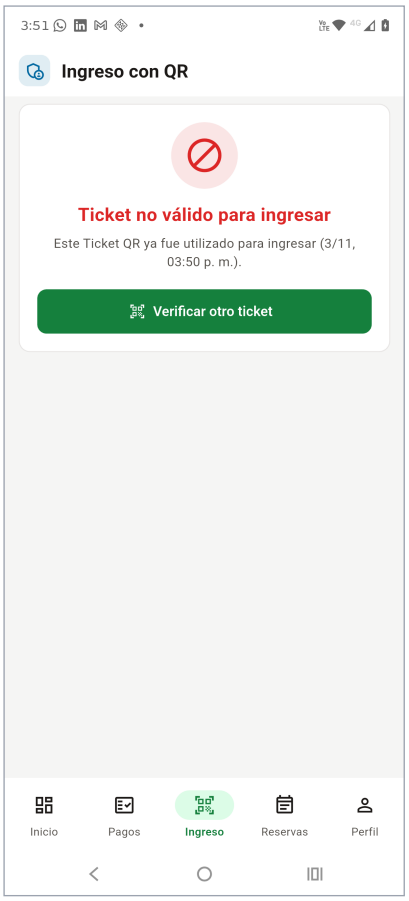


Figura 11
Segundo escaneo: QR ya utilizado (409).

3.8. Cancelación con la política de 24 horas

Sobre otra reserva confirmada (#4), el jugador elige *Cancelar reserva*. `POST /api/reservas/4/cancelar` aplica la política: con más de 24 horas de anticipación se devuelve el 100% de lo pagado. **Persistencia:** `estado = CANCELADA`, `monto_devolucion = 120`, `codigo_qr = NULL` y el motivo. Su QR revocado responde **409** si se intenta usar.



Figura 12
Reserva cancelada con reembolso del 100%.

4. Verificación de la persistencia en Supabase

Después de cada paso se consultó la base con las sentencias SQL de respaldo del guion. El estado observado quedó guardado en `docs/validaciones/capturas/evidencia-supabase-demo.json`:

Tabla 3. *Estados registrados en la base de datos durante la demostración.*

Registro	Momento	Estado persistido
reservas #19	Tras crear	PENDIENTE_PAGO, 2026-11-05 17:00-19:00, total 240
reserva_extras	Tras crear	Juego de Chalecos (x12) × 2 = 30
pagos #5	Tras enviar	PENDIENTE_REVISION, YAPE-8492014
Storage	Tras enviar	comprobantes (privado): reserva-19/voucher-1793738720689.jpg (63792 bytes, image/jpeg)
reservas #19	Tras enviar	EN_REVISION
pagos #5	Tras aprobar	APROBADO, revisado 2026-11-03T20:48:30.557Z
reservas #20	Paso 3.4	PENDIENTE_PAGO (20:00-21:00), reserva del competidor
reservas #4	Tras cancelar	CANCELADA, devolución 120 de 120, QR NULL
API	Segundo escaneo	409 Este Ticket QR ya fue utilizado para ingresar (3/11, 03:50 p. m.)
API	QR cancelado	409 El Ticket QR fue revocado porque la reserva #4 se encuentra CANCELADA.

5. Resumen de resultados

Tabla 4. *Resumen de los pasos, respuestas HTTP y resultados (todos cumplidos).*

Paso	Endpoint	Código	Resultado
3.1	GET /api/canchas?distrito=…&deporte=…	200	Catálogo filtrado desde la base
3.2	POST /api/reservas/cotizar	200	Total S/ 240 calculado en el servidor
3.3	POST /api/reservas	201	Reserva #19 PENDIENTE_PAGO con bloqueo
3.4	POST /api/reservas (mismo turno)	409	Un solo dueño por horario
3.5	POST /api/pagos/comprobante y /reservas/19/pagos	201	Voucher en Storage, reserva EN_REVISION
3.6	GET /api/pagos y PATCH /api/pagos/5/aprobar	200	Reserva CONFIRMADA y Ticket QR
3.7	POST /api/reservas/verificar-qr (dos veces)	200 / 409	Ingreso único
3.8	POST /api/reservas/4/cancelar	200	Reembolso 100% y QR revocado

6. Conclusiones

- La aplicación móvil consumió la API propia en todo el recorrido; ningún precio, disponibilidad ni cambio de estado se decidió en el cliente.
- Cada operación dejó un cambio verificable en PostgreSQL (reservas, extras y pagos) y en Supabase Storage (comprobante).
- Las reglas críticas se comprobaron en producción: exclusión mutua (409), roles (solo ADMIN aprueba y registra ingresos), QR de un solo uso (409 en el segundo escaneo) y reembolso según la política de 24 horas.
- La evidencia completa está en el repositorio: `docs/validaciones/DEMOSTRACION_TALLER3.md`, las capturas y el estado de la base en `evidencia-supabase-demo.json`.